PRACTICAL MACHINE LEARNING

Gradient Boosting

Learn it from the first prediction to the final model

1 Start with mean

2 Find errors

3 Train correction trees

4 Improve predictions

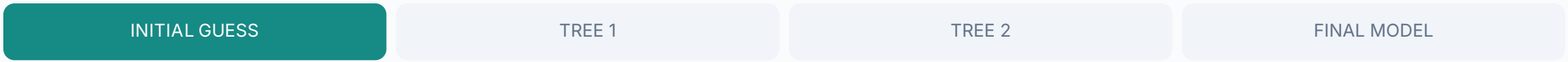
An end-to-end worked example • Regression • 2 trees

The one idea behind Gradient Boosting

One tree does not need to solve the whole problem. It only needs to improve the current prediction.

1 Make a prediction → 2 Find the errors → 3 Fit a tree to errors → 4 Add a small correction

REPEAT: the next tree corrects the NEW errors after the previous correction.



Our example: hours studied → exam marks

One feature and six students. Every calculation will use these same six rows.

Student	Hours studied (X)	Actual marks (y)
A	1	10
B	2	10
C	3	10
D	4	40
E	5	40
F	6	70

TASK

Given hours studied, predict the exam marks.

Our target is a number, so this is a REGRESSION problem.

Step 1 — Initial prediction = Mean(Actual Marks) = 30 for everyone

No decision tree has been built yet. This is just a constant model.

STEP 1

Hours	Actual marks	Initial prediction
1	10	30
2	10	30
3	10	30
4	40	30
5	40	30
6	70	30

$$F_0(x) = 30$$

The starting guess is the same for all hours.

Step 2 — Calculate residuals

Residual = Actual − Current prediction. The signs tell us the direction of correction.

STEP 2

$$\text{Residual} = y - \hat{y}$$

Student A: 10 − 30 = −20

Student F: 70 − 30 = +40

Hours	Actual	Predicted	Residual
1	10	30	-20
2	10	30	-20
3	10	30	-20
4	40	30	+10
5	40	30	+10
6	70	30	+40

Next tree will learn these six residuals.

Step 3 — Tree 1 is trained on residuals

The tree is NOT learning original marks. Its new target is the residual column.

STEP 3

Hours (X)	Actual marks (y)	Tree 1 target = residual
1	10	-20
2	10	-20
3	10	-20
4	40	+10
5	40	+10
6	70	+40

New supervised task

Hours → Residual

Regression tree leaves predict the mean residual within each leaf.

How does Tree 1 choose its split?

Try a split. Calculate the mean residual in each group and the total squared error.

Hours	Actual	Predicted	Residual
1	10	30	-20
2	10	30	-20
3	10	30	-20
4	40	30	+10
5	40	30	+10
6	70	30	+40

CANDIDATE: HOURS ≤ 3.5

Left: hours 1, 2, 3

Residuals: −20, −20, −20

Mean = −20 → SSE = 0

(-20-(20))+(-20-(20))+(-20-(20)) →
SSE = 0

Right: hours 4, 5, 6

Residuals: +10, +10, +40

Mean = +20

SSE = $(-10)^2 + (-10)^2 + 20^2 = 600$

Total SSE after split = 0 + 600 = 600

Why split at 3.5 hours?

A regression tree compares possible thresholds and selects the lowest squared-error split.

Possible threshold	SSE of residuals after split	Decision
1.5	2,520	Not best
2.5	1,800	Not best
3.5	600	BEST
4.5	1,125	Not best
5.5	1,080	Not best

Lowest total squared error → chosen split: Hours ≤ 3.5

Tree 1 learns two correction values

A depth-1 tree has only one split. Each leaf predicts its mean residual.



Important: These are CORRECTIONS, not exam-mark predictions.

Step 4 — Apply the learning rate

The tree suggests a correction. We choose how much of that correction to add.

STEP 4

New prediction = Old prediction + Learning rate × Tree correction

Learning rate = 0.5

We add HALF of Tree 1's suggested correction.

Why not always add 100%? Smaller steps can help control how aggressively the model fits.

Update all six predictions after Tree 1

Old prediction 30 + 0.5 × correction.

Hours	Actual	Old pred.	Tree 1 correction	Add 0.5 × correction	New prediction
1	10	30	-20	-10	20
2	10	30	-20	-10	20
3	10	30	-20	-10	20
4	40	30	+20	+10	40
5	40	30	+20	+10	40
6	70	30	+20	+10	40

Hours 6: 30 + 0.5 × (+20) = 40

Step 5 — The NEW residuals are different

We recompute errors after Tree 1. Tree 2 learns the remaining mistakes.

Hours	Actual marks	After Tree 1	NEW residual
1	10	20	-10
2	10	20	-10
3	10	20	-10
4	40	40	0
5	40	40	0
6	70	40	+30

Tree 2 target values: −10, −10, −10, 0, 0, +30

Step 6 — Train Tree 2 on NEW residuals

The best split now changes because the residual pattern has changed.



Left leaf mean = (−10 − 10 − 10 + 0 + 0) / 5 = −6

Step 7 — Update predictions after Tree 2

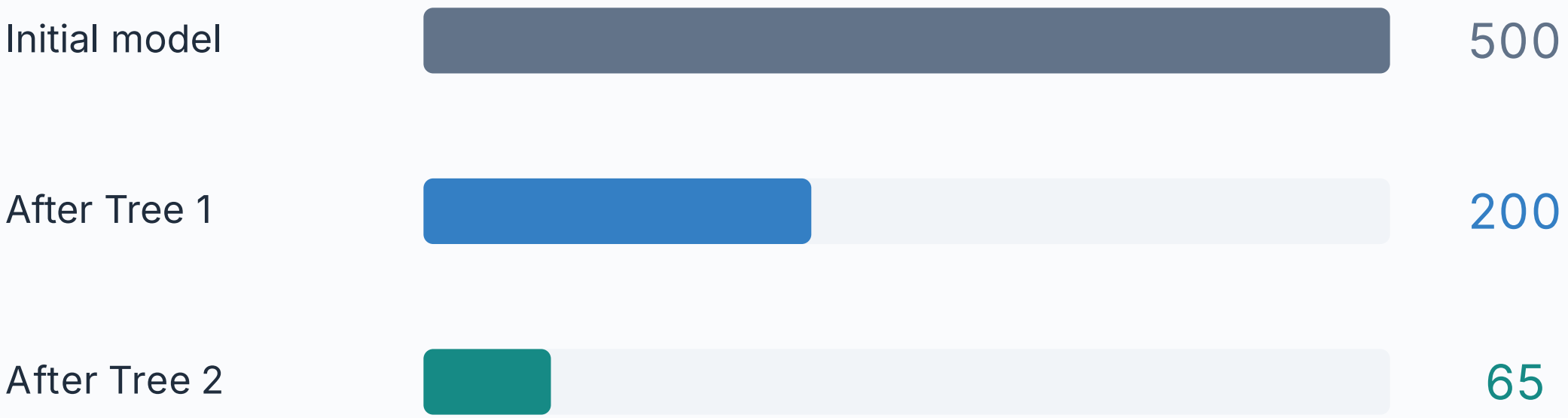
Again: New prediction = Previous prediction + 0.5 × Tree 2 correction.

Hours	Actual	After Tree 1	Tree 2 correction	Add 0.5 × correction	After Tree 2
1	10	20	-6	-3	17
2	10	20	-6	-3	17
3	10	20	-6	-3	17
4	40	40	-6	-3	37
5	40	40	-6	-3	37
6	70	40	+30	+15	55

Hours 6: 40 + 0.5 × 30 = 55

Did the model improve? Calculate MSE

We compare the same six actual marks after each stage.



500 → 200 → 65

How do the two trees combine?

At prediction time, add the scaled outputs of all trees to the initial constant.

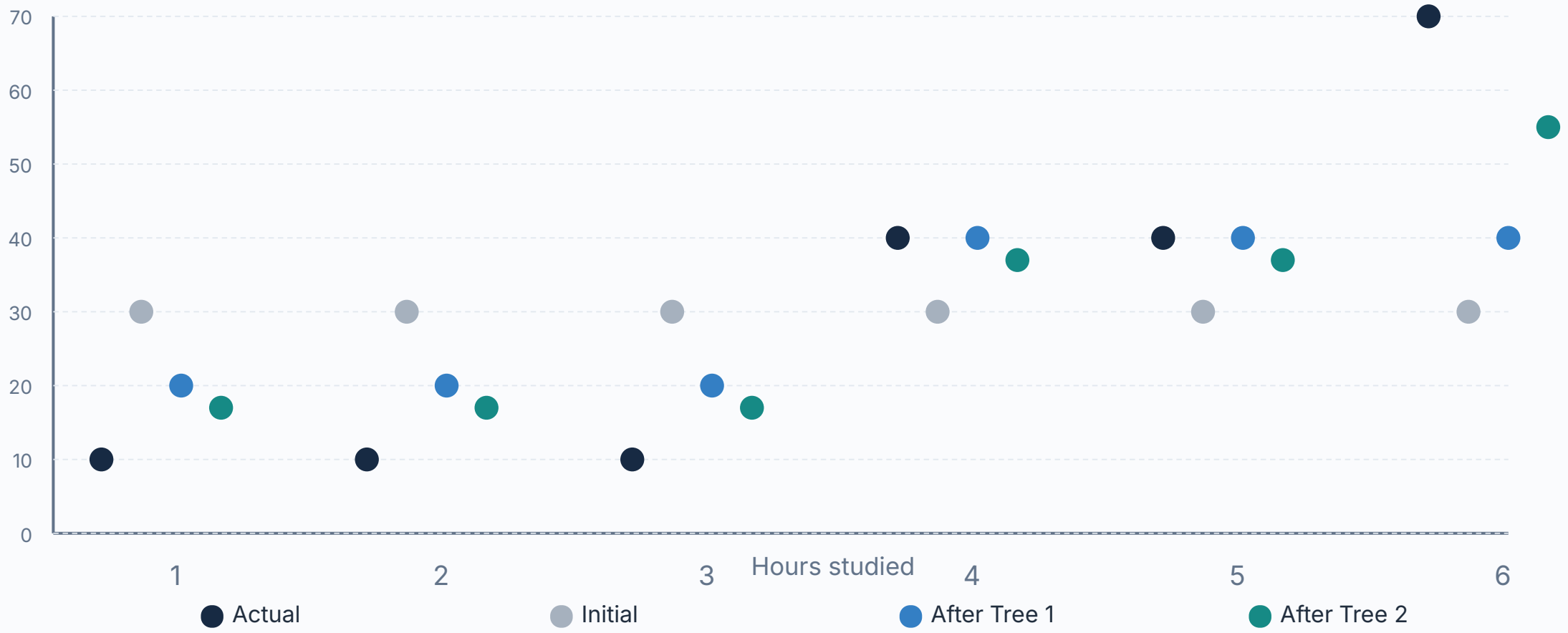
Final model = Initial value + 0.5 × Tree 1 + 0.5 × Tree 2

Initial model	Tree 1 at 6 hours	Tree 2 at 6 hours
30	+20	+30

At 6 hours: 30 + 0.5(20) + 0.5(30) = 55

Visualize predictions getting closer

For each student, the combined model changes as new trees are added.



Why is it called Gradient Boosting?

Squared-error loss connects the corrections we used to the direction of improvement.

Squared-error loss gets smaller when predictions move toward actual values.

For squared error: negative gradient = Actual – Prediction

So in our example, learning the residuals is learning the negative-gradient direction.

Other losses can produce different negative gradients; residuals are the special squared-error case.

Important scikit-learn parameters

Start with three parameters and add the others later.

n_estimators

Number of boosting stages

2 in our example

learning_rate

Fraction of each tree correction

0.5 in our example

max_depth

Complexity of each regression tree

1 (a decision stump)

min_samples_leaf

Smallest allowed leaf size

Regularizes the tree

subsample

Fraction of rows per stage

< 1 gives stochastic boosting

loss

Optimization objective

Squared error for this example

Smaller learning rates often need more trees; too many/deep trees can overfit.

Run the same two-tree example in Python

This code reproduces our hand-calculated predictions using scikit-learn.

```
import pandas as pd
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.metrics import mean_squared_error

df = pd.DataFrame({
    "Hours": [1, 2, 3, 4, 5, 6],
    "Score": [10, 10, 10, 40, 40, 70]
})
X, y = df[["Hours"]], df["Score"]

model = GradientBoostingRegressor(
    n_estimators=2, learning_rate=0.5,
    max_depth=1, random_state=42)
model.fit(X, y)
```

Verify predictions after every tree

staged_predict gives the combined model prediction after each boosting stage.

```
for step, pred in enumerate(model.staged_predict(X), start=1):
    print(f"After Tree {step}:", pred.round(0))
    print("MSE:", mean_squared_error(y, pred))

print("For 6 hours:", model.predict(pd.DataFrame({"Hours": [6]})))
```

After Tree 1: [20, 20, 20, 40, 40, 40] MSE: 200

After Tree 2: [17, 17, 17, 37, 37, 55] MSE: 65

For 6 hours: [55.]

Gradient Boosting in six steps

A single continuous workflow explains the entire algorithm.

01	Start with a constant prediction	For squared-error regression, use mean = 30.
02	Find residuals	Residual = actual − current prediction.
03	Fit Tree 1	One split; leaf means −20 and +20.
04	Update with the learning rate	Add 0.5 × Tree 1 correction.
05	Fit Tree 2 on remaining errors	Different residuals → different correction tree.
06	Add all contributions at prediction time	$F_2(x) = 30 + 0.5T_1(x) + 0.5T_2(x)$.

Big idea: Each new tree improves the existing model rather than starting from scratch.